Task B3 – Statement vs PreparedStatement vs Batch (8 Marks, K4)

Run every program yourself and attach screenshots linked to the test they show. Numbers labelled **sandbox** were measured by me on a 2-core cloud machine while preparing this answer, so repeat them on your computer and report your own figures. `<fill>` marks data you must collect.

Honest note: this task needs MySQL and the Connector/J driver. I had no MySQL server in my sandbox, so I **compiled** all the code below (it compiles cleanly) but did **not** run it against a database. There is therefore no measured data from me for B3; the tables are for your measurements, and I state the behaviour you should expect and why.

SQL script (`schema.sql`)

```
CREATE DATABASE IF NOT EXISTS buspass_test;      -- local test
database used only for this assignment
USE buspass_test;

DROP TABLE IF EXISTS bus_pass;
CREATE TABLE bus_pass (
    pass_id  VARCHAR(10)  NOT NULL PRIMARY KEY,
    reg_no   VARCHAR(12)  NOT NULL,
    name     VARCHAR(50)  NOT NULL,
    route_no INT          NOT NULL,
    amount   DECIMAL(10,2) NOT NULL,
    CHECK (amount > 0)
);
CREATE INDEX idx_route ON bus_pass(route_no);
```

Complete the Code – `PassLoader.java` (TODO 1, 2 and 3 filled in)

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;

public class PassLoader {
    /** Loads all passes in one transaction, in batches of 1,000 rows.
     *  Returns the number of rows loaded. */
    public static int loadAll(Connection con, List<String[]> passes)
            throws SQLException {
        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name,
route_no, amount) "
                + "VALUES (?, ?, ?, ?, ?)";
        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            int n = 0;
            for (String[] p : passes) {
```

```
                // TODO 1 (completed): set the five parameters, add to
batch, run batch every 1,000 rows
                ps.setString(1, p[0]);
                ps.setString(2, p[1]);
                ps.setString(3, p[2]);
                ps.setInt(4, Integer.parseInt(p[3]));
                ps.setDouble(5, Double.parseDouble(p[4]));
                ps.addBatch();
                n++;
                if (n % 1000 == 0) ps.executeBatch();
            }
            ps.executeBatch();
            // TODO 2 (completed): make all inserted rows permanent
and return the count
            con.commit();
            return n;
        } catch (SQLException | RuntimeException e) {
            // TODO 3 (completed): undo every insert made in this
transaction, then re-throw
            con.rollback();
            throw e;
        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

What each TODO does: **TODO 1** binds the five parameters (route as `int`, amount as `double`), calls `addBatch()`, and sends the batch with `executeBatch()` after every 1,000 rows, so the driver never holds more than 1,000 rows in memory. **TODO 2** `con.commit()` makes all inserts permanent at once and returns the count. **TODO 3** `con.rollback()` undoes every insert in this transaction, and then `throw e` re-throws so the caller knows it failed; `finally` restores the original auto-commit mode.

Run: `javac *.java && java -cp .:mysql-connector-j-8.x.x.jar BenchB3` (use `;` on Windows).

Experiment program (`BenchB3.java`)

```
import java.sql.*;
import java.util.*;

/** B3.1 - B3.3 experiments. Needs MySQL + schema.sql + Connector/J on
the class path. */
public class BenchB3 {
    static final String URL =
"jdbc:mysql://localhost:3306/buspass_test?
rewriteBatchedStatements=true";
    static List<String[]> gen(int n, String prefix) {
        List<String[]> l = new ArrayList<>();
        for (int i = 1; i <= n; i++) l.add(new String[]{prefix + i,
```

```
"19231" + i, "Student" + i, String.valueOf(i % 20 + 1), "4500.00"});
        return l;
    }
    static void clear(Connection c) throws SQLException { try
(Statement s = c.createStatement()) { s.executeUpdate("DELETE FROM
bus_pass"); } }

    static long viaStatement(Connection c, List<String[]> rows) throws
SQLException {
        long t = System.nanoTime();
        try (Statement st = c.createStatement()) {
            for (String[] p : rows)
                st.executeUpdate("INSERT INTO bus_pass VALUES ('" +
p[0] + "','" + p[1] + "','" + p[2] + "'," + p[3] + "," + p[4] + ")");
        }
        return (System.nanoTime() - t) / 1_000_000;
    }
    static long viaPrepared(Connection c, List<String[]> rows) throws
SQLException {
        long t = System.nanoTime();
        try (PreparedStatement ps = c.prepareStatement("INSERT INTO
bus_pass VALUES (?,?,?,?,?)")) {
            for (String[] p : rows) { ps.setString(1, p[0]);
ps.setString(2, p[1]); ps.setString(3, p[2]);
                ps.setInt(4, Integer.parseInt(p[3])); ps.setDouble(5,
Double.parseDouble(p[4])); ps.executeUpdate(); }
        }
        return (System.nanoTime() - t) / 1_000_000;
    }
    static long viaBatch(Connection c, List<String[]> rows) throws
SQLException {
        long t = System.nanoTime();
        PassLoader.loadAll(c, rows);
// batch + one transaction
        return (System.nanoTime() - t) / 1_000_000;
    }
    static int count(Connection c) throws SQLException {
        try (Statement s = c.createStatement(); ResultSet rs =
s.executeQuery("SELECT COUNT(*) FROM bus_pass")) { rs.next(); return
rs.getInt(1); }
    }
    public static void main(String[] args) throws SQLException {
        try (Connection c = DriverManager.getConnection(URL, "root",
"password")) {
            // B3.1 metadata check
            try (Statement s = c.createStatement(); ResultSet rs =
s.executeQuery("SELECT * FROM bus_pass LIMIT 1")) {
                ResultSetMetaData md = rs.getMetaData();
                for (int i = 1; i <= md.getColumnCount(); i++)
System.out.println(md.getColumnName(i) + " : " +
```

```
md.getColumnTypeName(i));
            }
            // B3.2 timing
            for (int n : new int[]{1000, 10000, 100000}) {
                List<String[]> rows = gen(n, "A");
                clear(c); long a = viaStatement(c, rows);
                clear(c); rows = gen(n, "B"); long b = viaPrepared(c,
rows);
                clear(c); rows = gen(n, "C"); long d = viaBatch(c,
rows);
                System.out.printf("n=%6d  Statement=%d ms  Prepared=%d
ms  Batch+transaction=%d ms%n", n, a, b, d);
            }
            // B3.3 forced failure half-way: row 5001 has an invalid
amount
            clear(c);
            List<String[]> bad = gen(10000, "F"); bad.get(5000)[4] =
"notANumber";
            try { PassLoader.loadAll(c, bad); } catch
(RuntimeException | SQLException e) { System.out.println("Transaction
failed: " + e); }
            System.out.println("Rows after failed TRANSACTION: " +
count(c));          // expected 0

            clear(c); c.setAutoCommit(true);
            try (PreparedStatement ps = c.prepareStatement("INSERT
INTO bus_pass VALUES (?,?,?,?,?)")) {
                for (String[] p : bad) {
// auto-commit: every row is committed at once
                    ps.setString(1, p[0]); ps.setString(2, p[1]);
ps.setString(3, p[2]);
                    ps.setInt(4, Integer.parseInt(p[3]));
ps.setDouble(5, Double.parseDouble(p[4])); ps.executeUpdate();
                }
            } catch (RuntimeException e) { System.out.println("Auto-
commit run failed: " + e); }
            System.out.println("Rows after failed AUTO-COMMIT run: " +
count(c));      // expected 5000
        }
    }
}
```

B3.1 – Correctness on small data

Insert 10 records with each approach (Statement, PreparedStatement, batch). Verify:

1. `SELECT COUNT(*) FROM bus_pass` returns 10 after each run (clear the table between runs).

2. `SELECT * FROM bus_pass LIMIT 1` and `ResultSetMetaData` (printed by `BenchB3`) show the columns and types. **Expected:** `pass_id : VARCHAR`, `reg_no : VARCHAR`, `name : VARCHAR`, `route_no : INT`, `amount : DECIMAL`.
3. Compare a few rows with the input (route is a number, amount has 2 decimals).

Approach	Rows found	Types correct?	Observation
Statement	`<fill>`	`<fill>`	
PreparedStatement	`<fill>`	`<fill>`	
PreparedStatement + batch	`<fill>`	`<fill>`	

B3.2 – Increase the data size (record the time in ms)

Records	(a) Statement	(b) PreparedStatement	(c) Batch + one transaction
1,000	`<fill>`	`<fill>`	`<fill>`
10,000	`<fill>`	`<fill>`	`<fill>`
100,000	`<fill>`	`<fill>`	`<fill>`

What you should expect (and explain in your report): (a) is the slowest: each row is a separate network round trip, the SQL text is parsed every time, and in auto-commit mode each row is also its own commit. (b) is a little faster or about the same, because the statement is prepared once, but it still has one round trip and one commit per row in auto-commit mode. (c) is by far the fastest (often 10–100 times faster for large n): rows are sent in groups (with `rewriteBatchedStatements=true` the MySQL driver rewrites a batch into multi-row inserts), and there is a single commit. All three grow roughly linearly with n; they differ in the cost per row.

B3.3 – Auto-commit vs transaction with a forced failure

`BenchB3` makes row 5,001 of 10,000 invalid (amount `notANumber`) and runs both ways.

Mode	Rows in `bus_pass` after the failure	Why
Single transaction (`PassLoader.loadAll`)	**0** (expected)	`rollback()` undoes the 5,000 rows inserted before the failure
Auto-commit, row by row	**5,000** (expected)	each of the first 5,000 rows was committed at once; the failure leaves a half-loaded table

Record the real counts: `<fill>` and screenshot `SELECT COUNT(*) FROM bus_pass`.

B3.4 – Engineering trade-off

- Statement: slowest, unsafe (SQL injection), re-parsed each time, so never use it with input values.
- PreparedStatement per row: safe and simple, but one round trip per row.
- Batch: fastest, safe, and the memory use is bounded by the batch size.
- Auto-commit: fast to write but leaves partial data after a failure; transaction: all-or-nothing, but holds locks and undo information until the end.

(a) Nightly import of 100,000 passes: `PreparedStatement` + `addBatch()/executeBatch()` (batches of ~1,000) inside **one transaction** (or a few big transactions with a record of progress), because it is the fastest, safe from injection, and a failure leaves the table unchanged so the job can be fixed and re-run. If the whole file is already clean, `LOAD DATA` is faster still. **(b) A single pass issued at the counter:** a single `PreparedStatement` insert with auto-commit (the statement is its own transaction). A batch gives no benefit for one row, but the parameters must still be bound with ?.

Alternative concept and complexity

One multi-row `INSERT` statement (`MultiRowInsert.java`) or **`LOAD DATA`** instead of `PreparedStatement` batches:

```
import java.sql.*;
import java.util.List;

/** Alternative: one multi-row INSERT per chunk (or LOAD DATA for the
whole file). */
public class MultiRowInsert {
    static void insertChunk(Connection c, List<String[]> chunk) throws
SQLException {
        StringBuilder sql = new StringBuilder("INSERT INTO
bus_pass(pass_id,reg_no,name,route_no,amount) VALUES ");
        for (int i = 0; i < chunk.size(); i++) sql.append(i == 0 ?
"(?,?,?,?,?)" : ",(?,?,?,?,?)");
        try (PreparedStatement ps =
c.prepareStatement(sql.toString())) {
            int k = 1;
            for (String[] p : chunk) { ps.setString(k++, p[0]);
ps.setString(k++, p[1]); ps.setString(k++, p[2]);
                ps.setInt(k++, Integer.parseInt(p[3]));
ps.setDouble(k++, Double.parseDouble(p[4])); }
            ps.executeUpdate();                                    //
one statement, one round trip for the whole chunk
        }
    }
    // LOAD DATA (server side, fastest, no per-row Java objects):
    // LOAD DATA LOCAL INFILE 'passes.csv' INTO TABLE bus_pass FIELDS
TERMINATED BY ','
    //   LINES TERMINATED BY '\n' (pass_id, reg_no, name, route_no,
```

```
amount);
}
```

- **Total time for n rows:** all three are O(n) (each row must be processed by the server). The constant differs: batched `PreparedStatement` = n/b round trips; multi-row insert = n/c statements for chunk size c, each parsed once; `LOAD DATA` = one statement and the server reads the file directly, with no per-row Java objects, usually the fastest by a large margin. Record your measurements for the three sizes `<fill>`.
- **Memory for a batch of b rows:** batch: the driver holds b parameter sets: **O(b)**. Multi-row insert: builds one SQL string (and a parameter list) with b value groups: **O(b)**, but the SQL text is larger, and MySQL limits it through `max_allowed_packet`, so chunks must stay small. `LOAD DATA`: **O(1)** in the Java program (the file is streamed by the driver or server), though the server needs its own buffers.
- **Trade-off:** `LOAD DATA` is fastest but needs file access and permission (`local_infile`), and gives less control over errors per row; multi-row insert is simple but needs chunking and building SQL; batches are the best general-purpose choice with good error handling.